SQL DATA ANALYSIS PROJECT

FOOD DELIVERY BANGALORE

PRESENTED BY
MUJAHID KHAN
MySQL · Data Analysis

Exploratory SQL analysis of 2,988 food delivery orders across Metropolitan, Urban & Semi-Urban Bangalore

KEY METRICS

2,988	**26 MIN**	**4.64**	**29%**
TOTAL ORDERS	AVG DELIVERY TIME	AVG RIDER RATING	LATE DELIVERY RATE

BUSINESS INSIGHTS

TRAFFIC IMPACT

Jam traffic causes the highest average delivery time of **30.61 min**, while Low traffic is just 21.50 min.

WEATHER DELAYS

Cloudy (28.68 min) & Foggy (28.28 min) weather conditions increase delivery times most. Sunny weather is fastest at 21.87 min.

CITY PERFORMANCE

Semi-Urban areas have the highest avg delivery time (~50 min). Metropolitan has the most orders at **2,300**.

FESTIVAL EFFECT

Orders during festivals average **44.41 min** vs 25.81 min on normal days — a **72% surge** in delivery time.

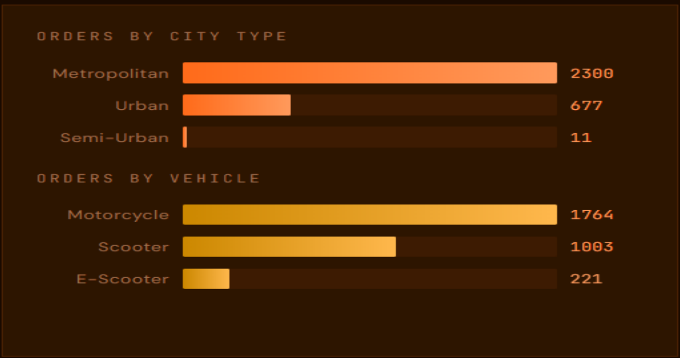
BEST VEHICLE

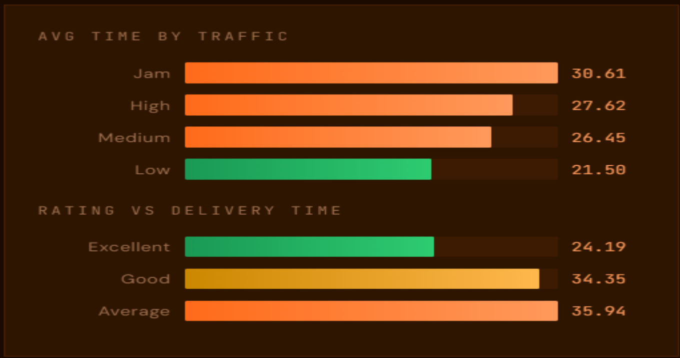
Scooters (24.21 min) outperform motorcycles (27.40 min) despite motorcycles handling the most deliveries (1,764).

DISTANCE FACTOR

Deliveries over **15 KM** average 30.17 min vs 22.31 min for under 5 KM — distance is a direct time driver.

DATA BREAKDOWN





FEATURED QUERY

```sql
-- Q1. Traffic Condition vs Delivery Time
SELECT Road_traffic_density,
       ROUND(AVG(delivery_time_mins), 2) AS AVERAGE_TIME
FROM food_delivery
GROUP BY Road_traffic_density
ORDER BY AVERAGE_TIME DESC;

-- Result: Jam → 30.61 min | High → 27.62 | Medium → 26.45 | Low → 21.50
```

First we check NULL values in column.

```sql
use food_delivery;

-- Check Null Values --

select * from food_delivery
where city is null;

-- Count Missing Cities --

select count(*) from food_delivery
where city = 'Nan';

```

```sql
-- Check the Percentage of Null or NaN values --
SELECT
ROUND(
COUNT(CASE WHEN City='NaN' THEN 1 END) * 100.0 / COUNT(*),
2
) AS missing_percentage
FROM food_delivery;
-- We have 3.08 % of null values so we can change the Mode for City Column --
```

Result Grid | Filter Rows:

missing_percentage
0.00

We have 3.08 % of null values so we can change the Mode for City Column.

Replacing with mode

```sql
-- Replacing With Mode in City Column --

SET SQL_SAFE_UPDATES = 0;

UPDATE food_delivery
SET City = 'Metropolitian'
WHERE City = 'NaN';
# Missing values in City (3.08%) were replaced with the mode (Metropolitian).
```

Missing values in City (3.08%) were replaced with the mode (Metropolitian).

```sql
UPDATE food_delivery
SET Festival = 'No'
WHERE Festival = 'NaN';

```

Some values in Festival column have NaN values so I change with No.

Total Orders

```sql
--  EXPLORATORY SQL ANALYSIS --

# 1 . Total Orders

SELECT COUNT(*) AS TOTAL_ORDERS
FROM food_delivery;
# Company processed 2988 orders.

```

	TOTAL_ORDERS
▶	2988

Company Processed 2988 Orders.

Average Delivery Time

```sql
# 2 Average Delivery Time

SELECT AVG(DELIVERY_TIME_MINS) AS Average_Delivery_Time
FROM food_delivery;
#Average delivery time is around 26 minutes.

```

	Average_Delivery_Time
▶	26.1319

Average Delivery Time is around 26 minutes.

Average Rider Rating

```sql
# 3 Average Rider_Rating

SELECT AVG(Delivery_person_Ratings) AS Average_Rider_Rating
FROM food_delivery;
# Average Rider Rating 4.6
```

Result Grid | Filter Rows:

Average_Rider_Rating
4.633935742971885

Orders By City

```sql
# 4 Orders By City

SELECT CITY , COUNT(*) AS Orders FROM food_delivery
GROUP BY CITY
ORDER BY Orders DESC;
# Most orders come from Metropolitian cities.
```

Result Grid | Filter Rows:

CITY	Orders
Metropolitian	2300
Urban	677
Semi-Urban	11

Most orders come from Metropolitian cities.

By Vehicle Type

```sql
# 5 Orders by Vehicle Type

SELECT Type_of_vehicle , COUNT(*) AS Orders
FROM FOOD_DELIVERY
GROUP BY Type_of_vehicle
ORDER BY Orders DESC;
# Motorcycles handle most deliveries.
```

Result Grid | Filter Rows:

Type_of_vehicle	Orders
motorcycle	1764
scooter	1003
electric_scooter	221

Motorcycles handle most deliveries.

By Traffic Density.

```sql
# 6 Orders by Traffic Density

SELECT ROAD_TRAFFIC_DENSITY , COUNT(*) AS Total_Orders
FROM food_delivery
GROUP BY ROAD_TRAFFIC_DENSITY
ORDER BY TOTAL_ORDERS DESC;
# Low and Jam traffic dominate deliveries.
```

Result Grid | Filter Rows: | Export:

ROAD_TRAFFIC_DENSITY	Total_Orders
Low	1050
Jam	942
Medium	714
High	282

Low and Jam traffic dominate deliveries.

Which Traffic Condition Causes Maximum Delivery Time?

```
--          BUISNESS QUESTIONS          --

# Q1. Which Traffic Condition Causes Maximum Delivery Time?

SELECT Road_traffic_density,
       ROUND(AVG(delivery_time_mins),2) AS AVERAGE_TIME
FROM food_delivery
GROUP BY Road_traffic_density
ORDER BY AVERAGE_TIME DESC;
# Jam traffic produces the highest delays.
```

Result Grid | Filter Rows: | Export:

Road_traffic_density	AVERAGE_TIME
Jam	30.61
High	27.62
Medium	26.45
Low	21.50

Jam traffic produces the highest delays .

Which weather condition causes delays?

```sql
# Q2. Which Weather Condition Causes Delays?

SELECT WEATHER_CLEAN , ROUND(AVG(DELIVERY_TIME_MINS),2) AS AVERAGE_TIME
FROM food_delivery
GROUP BY WEATHER_CLEAN
ORDER BY AVERAGE_TIME DESC;
# Cloudy and Foggy weather increase delivery time.

```

Result Grid | Filter Rows: | Export:

WEATHER_CLEAN	AVERAGE_TIME
Cloudy	28.68
Fog	28.28
Sandstorms	26.10
Windy	25.99
Stormy	25.83
Sunny	21.87

Cloudy & Foggy weather increases delivery time.

Top 10 Fastest delivery partners .

```sql
# Q3. Top 10 Fastest Delivery Partners.

SELECT DELIVERY_PERSON_ID , ROUND(AVG(DELIVERY_TIME_MINS),2) AS AVERAGE_TIME
FROM food_delivery
GROUP BY DELIVERY_PERSON_ID
ORDER BY AVERAGE_TIME
LIMIT 10;

```

Result Grid | Filter Rows: | Export:

DELIVERY_PERSON_ID	AVERAGE_TIME
BANGRES15DEL03	23.23
BANGRES15DEL02	23.28
BANGRES17DEL03	23.37
BANGRES11DEL03	24.08
BANGRES16DEL03	24.16
BANGRES06DEL03	24.56
BANGRES20DEL02	24.66
BANGRES03DEL03	24.75
BANGRES17DEL02	24.89
BANGRES010DEL02	24.95

Top Rated Riders.

```sql
# Q4. Top Rated Riders

SELECT DELIVERY_PERSON_ID, ROUND(AVG(DELIVERY_PERSON_RATINGS),2) AS AVERAGE_RATING
FROM food_delivery
GROUP BY DELIVERY_PERSON_ID
ORDER BY AVERAGE_RATING DESC
 ;
```

Result Grid | Filter Rows: | Export:

DELIVERY_PERSON_ID	AVERAGE_RATING
BANGRES010DEL02	4.65
BANGRES20DEL03	4.65
BANGRES11DEL01	4.65
BANGRES20DEL02	4.65
BANGRES14DEL01	4.64
BANGRES12DEL02	4.64
BANGRES06DEL03	4.64
BANGRES12DEL03	4.64
BANGRES03DEL02	4.64
BANGRES07DEL01	4.64
BANGRES04DEL03	4.63

Distance vs Delivery Time .

```sql
SELECT
CASE
WHEN distance_km <= 5 THEN '0-5 KM'
WHEN distance_km <= 10 THEN '5-10 KM'
WHEN distance_km <= 15 THEN '10-15 KM'
ELSE '15+ KM'
END AS distance_range,
ROUND(AVG(delivery_time_mins),2) AVERAGE_TIME
FROM food_delivery
GROUP BY distance_range
ORDER BY AVERAGE_TIME DESC;
# Longer distances directly increase delivery time.
```

Result Grid | Filter Rows: | Export:

distance_range	AVERAGE_TIME
15+ KM	30.17
10-15 KM	29.65
5-10 KM	23.86
0-5 KM	22.31

Longer distances directly increase delivery time.

Is Festival Impact on deliveries ?

```sql
# Q6. Festival Impact

SELECT FESTIVAL , AVG(DELIVERY_TIME_MINS) AS AVERAGE_TIME
FROM food_delivery
GROUP BY FESTIVAL
ORDER BY AVERAGE_TIME DESC
;
# Deliveries take longer during festivals.

```

Result Grid | Filter Rows: | Export:

FESTIVAL	AVERAGE_TIME
Yes	44.4118
No	25.8144

Yes , during festivals deliveries take longer time.

Which order type takes more time ?

```sql
# Q7. Which Order Type Takes More Time?

SELECT
    TYPE_OF_ORDER,
    ROUND(AVG(DELIVERY_TIME_MINS), 2) AS AVERAGE_TIME
FROM
    food_delivery
GROUP BY TYPE_OF_ORDER
ORDER BY AVERAGE_TIME DESC
;
```

Result Grid | Filter Rows: | Export:

TYPE_OF_ORDER	AVERAGE_TIME
Meal	26.62
Buffet	26.30
Drinks	25.93
Snack	25.69

Late delivery Percentage.

```sql
# Q8. Late Delivery Percentage

SELECT
    ROUND(SUM(IS_LATE) / COUNT(*), 2) AS LATE_DELIVERY_PCT
FROM
    FOOD_DELIVERY;
# Overall late delivery rate.

```

Result Grid | Filter Rows: | Export:

LATE_DELIVERY_PCT
0.29

Overall Late delivery rate .

City - Wise Delivery Performance .

```sql
# Q9. City-wise Delivery Performance

SELECT
    CITY, AVG(DELIVERY_TIME_MINS) AS AVERAGE_TIME
FROM
    FOOD_DELIVERY
GROUP BY CITY
ORDER BY AVERAGE_TIME DESC
;
# Semi-Urban has highest Average delivery time which is 50 min approx.
```

Result Grid | Filter Rows: | Export:

CITY	AVERAGE_TIME
Semi-Urban	50.1818
Metropolitan	26.9943
Urban	22.8109

Semi – Urban has Highest Average Delivery Time which is 50 Min Approx.

Best Vehicle Type .

```sql
# Q10. Best Vehicle Type

SELECT
    Type_of_vehicle,
    ROUND(AVG(delivery_time_mins), 2) AVERAGE_TIME
FROM
    food_delivery
GROUP BY Type_of_vehicle
ORDER BY AVERAGE_TIME;
# SCOOTER AND ELECTRIC SCOOTER ARE LESS AVERAGE TIME WHILE MOTORCYCLE TAKE MORE TIME.

```

Result Grid | Filter Rows: | Export:

Type_of_vehicle	AVERAGE_TIME
scooter	24.21
electric_scooter	24.70
motorcycle	27.40

Scooter and Electric Scooter are less average time while Motorcycle take more time.

Riders by Ranking .

```sql
# Q11. Rank Riders by Rating.

SELECT DELIVERY_PERSON_ID , DELIVERY_PERSON_RATINGS,
RANK()
  OVER
    (ORDER BY DELIVERY_PERSON_RATINGS DESC) AS RIDER_RNK
FROM food_delivery;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

DELIVERY_PERSON_ID	DELIVERY_PERSON_RATINGS	RIDER_RNK
BANGRES12DEL01	5	1
BANGRES08DEL02	5	1
BANGRES19DEL01	5	1
BANGRES04DEL01	5	1
BANGRES14DEL01	5	1
BANGRES15DEL01	5	1
BANGRES14DEL02	5	1

Which is Top Performing Riders ?

```sql
# Q12. Which is Top Peroforming Riders ?

WITH RIDER_PERFORMANCE AS (
SELECT DELIVERY_PERSON_ID , AVG(DELIVERY_TIME_MINS) AS AVERAGE_TIME
FROM FOOD_DELIVERY
GROUP BY DELIVERY_PERSON_ID
ORDER BY AVERAGE_TIME DESC
)

SELECT * FROM RIDER_PERFORMANCE
WHERE AVERAGE_TIME < 25;
```

Result Grid | Filter Rows: | Export:

DELIVERY_PERSON_ID	AVERAGE_TIME
BANGRES20DEL03	24.9744
BANGRES010DEL02	24.9474
BANGRES17DEL02	24.8929
BANGRES03DEL03	24.7500
BANGRES20DEL02	24.6600
BANGRES06DEL03	24.5641
BANGRES16DEL03	24.1591

Find Riders Better Than Average .

```sql
# Q13. Find Riders Better Than Average

SELECT
    Delivery_person_ID,
    ROUND(AVG(delivery_time_mins), 2) AS AVERAGE_TIME
FROM
    food_delivery
GROUP BY Delivery_person_ID
HAVING AVG(delivery_time_mins) < (SELECT
        AVG(delivery_time_mins)
    FROM
        food_delivery);
```

Result Grid | Filter Rows: | Export:

Delivery_person_ID	AVERAGE_TIME
BANGRES18DEL02	25.07
BANGRES17DEL02	24.89
BANGRES17DEL01	26.11
BANGRES05DEL03	25.89
BANGRES11DEL01	26.07
BANGRES03DEL03	24.75
BANGRES15DEL02	23.28

Top Cities by Orders.

```sql
# Q14 Top Cities by Orders.

SELECT
    CITY, COUNT(*) AS TOTAL_ORDERS
FROM
    food_delivery
GROUP BY CITY
ORDER BY TOTAL_ORDERS DESC;
# Metropolitan cities have the highest order at 2,300, while semi-urban areas have the lowest order at 11.

```

Result Grid | Filter Rows: | Export:

CITY	TOTAL_ORDERS
Metropolitan	2300
Urban	677
Semi-Urban	11

Metropolitian City have the highest order 2300.

Rating VS Delivery Time .

```sql
# Q15 Rating vs Delivery Time.

SELECT
    CASE
        WHEN DELIVERY_PERSON_RATINGS >= 4.5 THEN 'EXCELLENT'
        WHEN DELIVERY_PERSON_RATINGS >= 4.0 THEN 'GOOD'
        ELSE 'AVERAGE'
    END AS RIDER_CATEGORY,
    AVG(DELIVERY_TIME_MINS) AS AVERAGE_TIME
FROM
    FOOD_DELIVERY
GROUP BY RIDER_CATEGORY;
```

Result Grid | Filter Rows:

RIDER_CATEGORY	AVERAGE_TIME
EXCELLENT	24.1872
GOOD	34.3485
AVERAGE	35.9368

Monthly Trend .

```sql
# Q16 Monthly Trend .

SELECT
    MONTH(order_date) AS MONTH, COUNT(*) AS ORDERS
FROM
    food_delivery
GROUP BY month;
# March reached 2499 Orders , but April dropped to the lowest point at 489 Orders.
```

Result Grid | Filter Rows: | Export:

MONTH	ORDERS
3	2499
4	489

March month reached 2499 orders , but April dropped.